



NeoBank 360

Project Q & A — Viva / Review Preparation

A curated set of questions and model answers covering the architecture, security, core banking, lending, and design decisions behind NeoBank 360 — prepared for project review and viva.

1 · Project Overview & Architecture

Q1. What is NeoBank 360 and what problem does it solve?

NeoBank 360 is a unified digital banking platform that brings accounts, fund transfers, lending, budgeting, bill payment and rewards into a single secure application. It solves the fragmentation problem in everyday banking, where these capabilities usually live in disconnected silos, by giving customers one real-time view of their money and giving administrators one console for loan decisions, analytics and audit.

Q2. Describe the overall architecture.

It is a layered architecture across four tiers. The **client** is an Angular single-page app that attaches a JWT to every request and uses route guards. The **security** layer is a stateless Spring Security filter chain fronted by a custom JwtAuthFilter. The **application** layer is a set of REST controllers delegating to a transactional service layer, with an AOP aspect wrapping every controller call. The **data** layer is Spring Data JPA over MySQL, with the audit log as a dedicated store.

Q3. Why did you choose a modular monolith instead of microservices?

For a project of this scope a modular monolith gives clean module boundaries (auth, accounts, transfers, loans, bills, budgets, rewards, analytics) without the operational overhead of distributed systems — no network hops between services, simpler transactions, and a single deployable. The internal layering keeps it ready to decompose into microservices later if scale demands it.

Q4. How does a request flow through the system end to end?

The Angular interceptor adds the Bearer token; the request reaches the JwtAuthFilter, which validates the token and loads the principal via UserDetailsService; the Spring Security chain authorises it (for example, admin routes require the ADMIN role); the controller delegates to a service method that runs inside a transaction; the service uses JPA repositories to read and write MySQL; and the AOP aspect records the call into the audit log on the way out.

2 · Security & Authentication

Q1. How is authentication implemented?

Authentication is stateless using JSON Web Tokens. On login the server verifies the BCrypt-hashed password and issues an HS256-signed JWT carrying the user's identity. The client stores it and sends it as a Bearer token on each request. A custom JwtAuthFilter extracts and validates the token, then populates the Spring SecurityContext so downstream authorization can run.

Q2. Why stateless JWT instead of server-side sessions?

Stateless tokens mean the server keeps no session state, which makes the API horizontally scalable and simpler to reason about. Each request carries its own proof of identity, so any instance can serve it. The trade-off is that tokens can't be trivially revoked before expiry, which we mitigate with a sensible expiry window.

Q3. How is authorization enforced between customers and admins?

Roles are modelled as an enum (CUSTOMER and ADMIN). The Spring Security chain restricts admin endpoints (paths under /admin/**) to the ADMIN role, while other authenticated endpoints are available to signed-in users. On the front end, route guards (auth, admin and guest) prevent unauthorised navigation, giving defence in depth.

Q4. How are passwords protected?

Passwords are never stored in plain text. They are hashed with BCrypt, a deliberately slow adaptive hash with a per-password salt, so even identical passwords produce different hashes and brute-forcing is expensive.

Q5. What security hardening is applied to the filter chain?

The chain is stateless (no HTTP session is created), CSRF is disabled because the API is token-based rather than cookie-based, CORS is configured for the Angular origin, public endpoints like login and registration are explicitly permitted, and everything else requires authentication. The JwtAuthFilter is registered before the standard username/password filter.

3 · Core Banking & Transactions

Q1. How do fund transfers guarantee consistency?

Transfers run inside a single database transaction annotated with @Transactional. The service validates that the source account belongs to the user and has sufficient balance, then debits the source and credits the destination together. Because both legs are in one transaction, either both succeed or both roll back — the books can never end up half-updated.

Q2. What happens if the source account has insufficient balance?

The service compares the balance to the requested amount before moving any money and rejects the transfer with a clear error if funds are insufficient, so no partial debit occurs.

Q3. How is transaction history maintained?

Every money movement writes a row to the transaction ledger capturing the type, amount, the running balance after the movement, a description and a timestamp, linked to the account. This gives each account a complete, ordered history.

4 · Loan & EMI Engine

Q1. Explain the EMI formula you use.

We use the standard reducing-balance EMI: $E = P \cdot r \cdot (1+r)^n / ((1+r)^n - 1)$, where P is the principal, r is the monthly interest rate (annual rate $\div 12 \div 100$) and n is the tenure in months. It produces a level instalment where the interest portion shrinks and the principal portion grows over the tenure.

Q2. How is the repayment schedule generated?

When an admin approves an application, the EMI amount is computed and a full amortisation schedule is generated — one row per instalment with its due date, the EMI amount, and the split between principal and interest components, each starting in a pending status.

Q3. How do you avoid rounding errors across the schedule?

Monetary values are rounded to two decimals with HALF_UP rounding. Because per-instalment rounding can leave a tiny residual, the final instalment absorbs the remaining outstanding principal, so the schedule closes exactly to the last cent.

Q4. How are loan products defined and validated?

An admin defines loan products with a minimum and maximum amount, an annual interest rate and a list of allowed tenures. When a customer applies, the requested amount and tenure are validated against the product's configuration before the application can proceed.

Q5. What is the loan lifecycle?

Product is configured by an admin → customer submits an application → admin approves or rejects with remarks → on approval a loan account is created, the EMI is computed and the repayment schedule is generated → repayments are tracked per instalment with paid, pending or overdue status.

5 · Budgeting, Bills & Rewards

Q1. How does budgeting work?

Customers set monthly spending limits per category. The insights engine aggregates their actual outflow against those limits, giving a clear picture of where money is going and how much budget remains.

Q2. How is bill status, including overdue, determined?

Each bill stores a biller, amount and due date and carries a paid, pending or overdue status. Overdue is determined by comparing the due date against the current date, so overdue bills surface automatically without manual intervention.

Q3. What is the rewards feature?

Each user has a rewards balance of points that reflects their activity on the platform, stored alongside the time it was last updated.

6 · Auditing, Analytics & Design Decisions

Q1. How is auditing implemented and why use AOP?

An AOP aspect wraps every controller method. Around each call it records the endpoint, HTTP method, response status, execution time, the acting user and any error, writing them to the system audit log. Using AOP keeps this cross-cutting concern out of the business logic entirely — controllers and services stay focused on their job, and auditing is applied uniformly and automatically.

Q2. What does the admin analytics dashboard show?

It surfaces real-time platform activity — transaction and loan trends and user-level analytics — rendered with Chart.js, plus operational controls for approving loan applications and managing users.

Q3. Why separate DTOs from entities?

DTOs decouple the API contract from the persistence model. They let us validate and shape exactly what crosses the wire, avoid leaking internal fields (like password hashes), and prevent accidental changes to the database schema from breaking clients or vice versa.

Q4. How would you scale or extend the platform?

Immediate steps include adding scheduled background jobs to auto-mark overdue EMIs and bills and send reminders. Longer term: decomposing the monolith into microservices, adding ML-driven spend insights and anomaly detection, native mobile apps on the same API, biometric and TOTP two-factor authentication, and open-banking APIs for third-party integrations.

Q5. What were the main engineering challenges and learnings?

Getting financial precision right (rounding plus a residual-absorbing final EMI), guaranteeing atomicity in transfers, mapping endpoints cleanly onto role-based access with stateless JWT, keeping the controller-service-repository layering disciplined with DTOs at the edges, and using AOP to add auditing without polluting business logic. Coordinating the work across the team also sharpened our Git workflow.